

Calibration of the asymptotic item-restscore test

Null simulation of `iarm::item_restscore()` under a true Rasch model

Magnus Johansson

2026-10-05

Table of contents

Purpose	1
Design	2
Item parameters	2
Data generation	3
One replication	3
Run	3
Results	5
Per-item rejection rates	5
Family-wise rejection	6
Where the SE goes wrong	6
Per-item rates by item location	7
Distribution of z	8
Interpretation	9
Reference	10

Purpose

`RMitemRestscore()` flags items using the p-value from `iarm::item_restscore()`. The test statistic is

$$z_i = \frac{\hat{\gamma}_i^{\text{obs}} - \hat{\gamma}_i^{\text{exp}}}{\text{ASE}(\hat{\gamma}_i^{\text{obs}})},$$

referred to the standard normal, two-sided, then adjusted over the k items (BH by default). The observed gamma is Goodman-Kruskal gamma for the item $\times$ restscore table. The expected gamma is computed from the CML item parameters and the observed score distribution. The ASE is the delta-method standard error that `vcdExtra::GKgamma()` returns for the observed table.

Two features of this construction give reasons to check calibration directly instead of assuming it:

1. **The expected gamma is treated as known.** It is a function of estimated thresholds, and it is estimated from the same data as the observed gamma. The ASE ignores both the estimation uncertainty and the correlation between the two terms. The sign of the resulting bias cannot be worked out without simulating it.
2. **The ASE is the delta-method SE computed under the alternative.** van der Ark and van Aert (2015) found it had the worst coverage of four CI types for bivariate gamma, but only for tables up to 5×5 at $N \leq 100$. The item $\times$ restscore table is larger (4×28 for nine four-category items), with sparse cells in the tails of the restscore.

The partial gamma LD and DIF functions already have simulation-based cutoffs. Item-restscore does not. `RMitemRestscoreBoot()` resamples the data and counts how often each item is flagged, which describes stability and not calibration.

An earlier simulation hinted at this. It reported item-restscore as well calibrated from per-item false positive rates, but counted as the share of replications with at least one BH-flagged item, its small-

sample rate was above 5 percent. That study is not reported here, so this is the motivation for the check, not evidence.

Design

- **Item sets.** (a) 20 dichotomous items with locations evenly spaced on $[-2, 2]$, as in `sx2_comparison_v2.qmd`. (b) 9 polytomous items with four categories, locations evenly spaced on $[-1.5, 1.5]$, and thresholds at location ± 1.2 and location $+0$, as a PHQ-9-sized scale.
- **Persons.** $\theta \sim N(\mu, 1.5^2)$, with $\mu = 0$ (targeted) or $\mu = 1.5$ (mistargeted), since sparse restscore tails are a plausible source of miscalibration.
- **Sample sizes.** 150, 300, 600, 1000, 2000.
- **Replications.** 4000 per cell. The Monte Carlo SE of a 5 percent rate is then 0.34 points.
- **Fitting.** Same as the package: `psychotools::pcmodel()`, then `iarm::item_restscore()`. A replication with an unobserved category (or an item with zero variance) is discarded and counted, since the package would not fit it either.

`iarm` returns its results rounded to four decimals, so z is recomputed from the rounded observed, expected and SE columns and the p-values from z . With SEs around .02 to .05 the rounding moves z by at most about .003.

For each item and replication the observed gamma, expected gamma and ASE are stored. Calibration is summarised in three ways.

- Per-item rejection at two-sided $\alpha = .05$, and separately in each tail at .025.
- Family-wise rejection (any item flagged) with no adjustment, BH as in the package default, and Bonferroni.
- A decomposition of the SE. If the ASE were correct for the test statistic, the empirical SD of $\hat{\gamma}^{\text{obs}} - \hat{\gamma}^{\text{exp}}$ across replications would equal the root mean squared ASE. The ratio $\text{SD}(\hat{\gamma}^{\text{obs}}) / \text{RMS}(\text{ASE})$ checks the ASE as an SE for the observed gamma alone. Comparing the two ratios separates point 1 from point 2 above.

```
suppressMessages({
  library(psychotools)
  library(iarm)
  library(parallel)
  library(dplyr)
  library(tidyr)
  library(ggplot2)
})
options(rgl.useNULL = TRUE)

R_REPS <- 4000
NS <- c(150, 300, 600, 1000, 2000)
MUS <- c(0, 1.5)
THETA_SD <- 1.5
NCORES <- 10
RES_FILE <- "restscore_asymptotic_null.rds"
```

Item parameters

```
item_sets <- list(
  dich = as.list(seq(-2, 2, length.out = 20)),
  poly = lapply(seq(-1.5, 1.5, length.out = 9), function(l) l + c(-1.2, 0, 1.2))
)

data.frame(
```

```

set = c("dich", "poly"),
items = lengths(item_sets),
categories = c(2, 4),
location_range = c("-2 to 2", "-1.5 to 1.5")
)

```

	set	items	categories	location_range
dich	dich	20	2	-2 to 2
poly	poly	9	4	-1.5 to 1.5

Data generation

Partial credit model, vectorised over persons. `thr` is one threshold vector per item. A dichotomous item is a PCM item with one threshold.

```

sim_pcm <- function(theta, thr) {
  n <- length(theta)
  sapply(thr, function(d) {
    m <- length(d)
    eta <- outer(theta, 0:m) -
      matrix(c(0, cumsum(d)), n, m + 1, byrow = TRUE)
    p <- exp(eta - apply(eta, 1, max))
    p <- p / rowSums(p)
    cp <- p %*% upper.tri(diag(m + 1), diag = TRUE)
    rowSums(runif(n) > cp)
  })
}

```

One replication

```

one_rep <- function(rep, set, n, mu) {
  set.seed(1e7 * match(set, names(item_sets)) + 1e5 * match(mu, MUS) +
    10 * n + rep)
  thr <- item_sets[[set]]
  d <- as.data.frame(sim_pcm(rnorm(n, mu, THETA_SD), thr))
  names(d) <- paste0("I", seq_along(thr))

  # every category observed in every item, as the CML fit requires
  full <- all(mapply(function(x, m) all(tabulate(x + 1, m + 1) > 0),
    d, lengths(thr)))
  if (!full) return(NULL)

  res <- tryCatch({
    fit <- psychotools::pcmodel(d, hessian = FALSE)
    r <- iarm::item_restscore(fit, p.adj = "none")
    matrix(as.numeric(r[seq_along(thr), c("observed", "expected", "se")]),
      ncol = 3, dimnames = list(NULL, c("obs", "exp", "se")))
  }, error = function(e) NULL)
  res
}

```

Run

Results are cached. Delete `restscore_asymptotic_null.rds` to force a re-run.

```

grid <- expand.grid(rep = seq_len(R_REPS), n = NS, mu = MUS,
                  set = names(item_sets), stringsAsFactors = FALSE)

if (file.exists(RES_FILE)) {
  sim <- readRDS(RES_FILE)
} else {
  t0 <- Sys.time()
  res <- mclapply(seq_len(nrow(grid)),
                 function(i) one_rep(grid$rep[i], grid$set[i],
                                     grid$n[i], grid$mu[i]),
                 mc.cores = NCORES, mc.preschedule = TRUE)
  sim <- list(res = res, grid = grid, item_sets = item_sets,
             elapsed_min = as.numeric(Sys.time() - t0, units = "mins"))
  saveRDS(sim, RES_FILE)
}
cat("elapsed:", round(sim$elapsed_min, 1), "min\n")

```

```
elapsed: 9.9 min
```

```

ok <- !vapply(sim$res, is.null, logical(1))

sim$grid |>
  mutate(ok = ok) |>
  group_by(set, mu, n) |>
  summarise(discarded = sum(!ok), .groups = "drop") |>
  filter(discarded > 0)

```

set	mu	n	discarded
poly	0.0	150	1
poly	1.5	150	1126
poly	1.5	300	224
poly	1.5	600	8

```

item_loc <- lapply(sim$item_sets, function(s) vapply(s, mean, numeric(1)))

long <- do.call(rbind, lapply(which(ok), function(i) {
  g <- sim$grid[i, ]
  m <- sim$res[[i]]
  data.frame(set = g$set, mu = g$mu, n = g$n, rep = g$rep,
            item = seq_len(nrow(m)),
            location = item_loc[[g$set]], m)
})) |>
  mutate(diff = obs - exp,
         z = diff / se,
         p = 2 * pnorm(-abs(z)))

```

Results

Per-item rejection rates

Nominal 5.0 percent two-sided, 2.5 percent in each tail. **over** is observed gamma above expected (the package labels this overfit), **under** is below.

An observed gamma of exactly ± 1 has an ASE of zero, which gives an infinite z and $p = 0$. The package flags such an item, so it counts as a rejection here, but it is left out of the mean and SD of z . **se_zero** counts these cases.

```
long |>
  group_by(set, mu, n) |>
  summarise(two_sided = 100 * mean(p < .05),
            over = 100 * mean(z > qnorm(.975)),
            under = 100 * mean(z < qnorm(.025)),
            mean_z = mean(z[is.finite(z)]),
            sd_z = sd(z[is.finite(z)]),
            se_zero = sum(se == 0),
            .groups = "drop") |>
  mutate(across(two_sided:under, \(x) round(x, 2)),
         across(c(mean_z, sd_z), \(x) round(x, 3)))
```

set	mu	n	two_sided	over	under	mean_z	sd_z	se_zero
dich	0.0	150	4.74	4.11	0.64	0.153	0.970	0
dich	0.0	300	3.78	2.99	0.79	0.104	0.932	0
dich	0.0	600	3.56	2.58	0.98	0.072	0.921	0
dich	0.0	1000	3.36	2.34	1.02	0.057	0.921	0
dich	0.0	2000	3.08	1.94	1.15	0.039	0.907	0
dich	1.5	150	6.29	5.66	0.63	0.212	1.135	2
dich	1.5	300	4.71	4.01	0.70	0.130	0.981	0
dich	1.5	600	3.94	3.08	0.86	0.089	0.942	0
dich	1.5	1000	3.76	2.70	1.06	0.067	0.934	0
dich	1.5	2000	3.37	2.27	1.09	0.046	0.922	0
poly	0.0	150	2.94	2.67	0.27	0.149	0.862	0
poly	0.0	300	2.40	1.98	0.42	0.106	0.842	0
poly	0.0	600	2.03	1.59	0.44	0.073	0.837	0
poly	0.0	1000	1.89	1.35	0.54	0.057	0.833	0
poly	0.0	2000	1.89	1.24	0.65	0.040	0.831	0
poly	1.5	150	3.16	2.92	0.24	0.152	0.874	0
poly	1.5	300	2.59	2.23	0.36	0.106	0.856	0
poly	1.5	600	2.07	1.62	0.44	0.072	0.835	0
poly	1.5	1000	2.14	1.61	0.52	0.058	0.838	0
poly	1.5	2000	2.04	1.36	0.68	0.041	0.840	0

Family-wise rejection

Proportion of replications with at least one flagged item. Under a complete null, BH and Bonferroni both control this at 5 percent. The unadjusted column shows what the per-item test implies across 9 or 20 items.

```
fw <- long |>
  group_by(set, mu, n, rep) |>
  summarise(none = any(p < .05),
            bh = any(p.adjust(p, "BH") < .05),
            bonferroni = any(p.adjust(p, "bonferroni") < .05),
            .groups = "drop") |>
  group_by(set, mu, n) |>
  summarise(reps = n(),
            across(c(none, bh, bonferroni), \(x) round(100 * mean(x), 2)),
            .groups = "drop")
fw
```

set	mu	n	reps	none	bh	bonferroni
dich	0.0	150	4000	63.38	13.45	13.38
dich	0.0	300	4000	54.93	6.20	6.15
dich	0.0	600	4000	51.95	4.05	4.03
dich	0.0	1000	4000	49.30	2.72	2.67
dich	0.0	2000	4000	45.80	2.30	2.28
dich	1.5	150	4000	73.78	30.50	30.18
dich	1.5	300	4000	63.05	13.88	13.80
dich	1.5	600	4000	56.03	7.38	7.32
dich	1.5	1000	4000	53.12	4.85	4.82
dich	1.5	2000	4000	49.33	3.22	3.15
poly	0.0	150	3999	24.08	4.23	4.18
poly	0.0	300	4000	19.68	2.17	2.17
poly	0.0	600	4000	16.92	1.80	1.77
poly	0.0	1000	4000	15.40	0.92	0.92
poly	0.0	2000	4000	15.68	1.10	1.10
poly	1.5	150	2874	25.96	5.11	5.08
poly	1.5	300	3776	21.00	3.02	2.99
poly	1.5	600	3992	17.16	1.65	1.65
poly	1.5	1000	4000	17.50	1.18	1.12
poly	1.5	2000	4000	16.83	1.60	1.60

Where the SE goes wrong

`sd_diff / rms_se` near 1 means the ASE is the right SE for the test statistic. `sd_obs / rms_se` near 1 means it is the right SE for the observed gamma alone. `cor_obs_exp` is the correlation between observed and expected gamma across replications, averaged over items.

```

se_tab <- long |>
  group_by(set, mu, n, item) |>
  summarise(sd_diff = sd(diff), sd_obs = sd(obs), sd_exp = sd(exp),
            rms_se = sqrt(mean(se^2)), cor_obs_exp = cor(obs, exp),
            .groups = "drop") |>
  group_by(set, mu, n) |>
  summarise(sd_diff_over_rms_se = mean(sd_diff / rms_se),
            sd_obs_over_rms_se = mean(sd_obs / rms_se),
            sd_exp_over_rms_se = mean(sd_exp / rms_se),
            cor_obs_exp = mean(cor_obs_exp),
            .groups = "drop") |>
  mutate(across(where(is.double) & !c(mu), \(x) round(x, 3)))
se_tab

```

set	mu	n	sd_diff_over_rms_se	sd_obs_over_rms_se	sd_exp_over_rms_se	cor_obs_exp
dich	0.0	150	0.914	1.008	0.428	0.422
dich	0.0	300	0.907	1.000	0.421	0.421
dich	0.0	600	0.909	1.001	0.420	0.419
dich	0.0	1000	0.913	1.003	0.413	0.412
dich	0.0	2000	0.904	1.000	0.428	0.428
dich	1.5	150	0.926	1.012	0.407	0.401
dich	1.5	300	0.919	1.004	0.401	0.398
dich	1.5	600	0.915	1.001	0.402	0.402
dich	1.5	1000	0.919	1.007	0.407	0.406
dich	1.5	2000	0.915	0.998	0.396	0.396
poly	0.0	150	0.830	1.000	0.563	0.558
poly	0.0	300	0.827	1.005	0.571	0.568
poly	0.0	600	0.828	0.996	0.554	0.556
poly	0.0	1000	0.828	0.997	0.559	0.556
poly	0.0	2000	0.829	0.999	0.559	0.557
poly	1.5	150	0.829	0.999	0.553	0.555
poly	1.5	300	0.834	0.997	0.545	0.547
poly	1.5	600	0.825	0.988	0.541	0.550
poly	1.5	1000	0.831	0.998	0.552	0.551
poly	1.5	2000	0.836	1.002	0.547	0.549

Per-item rates by item location

If sparse restscores matter, items far from the sample mean should be the worst calibrated, and more so under mistargeting.

```

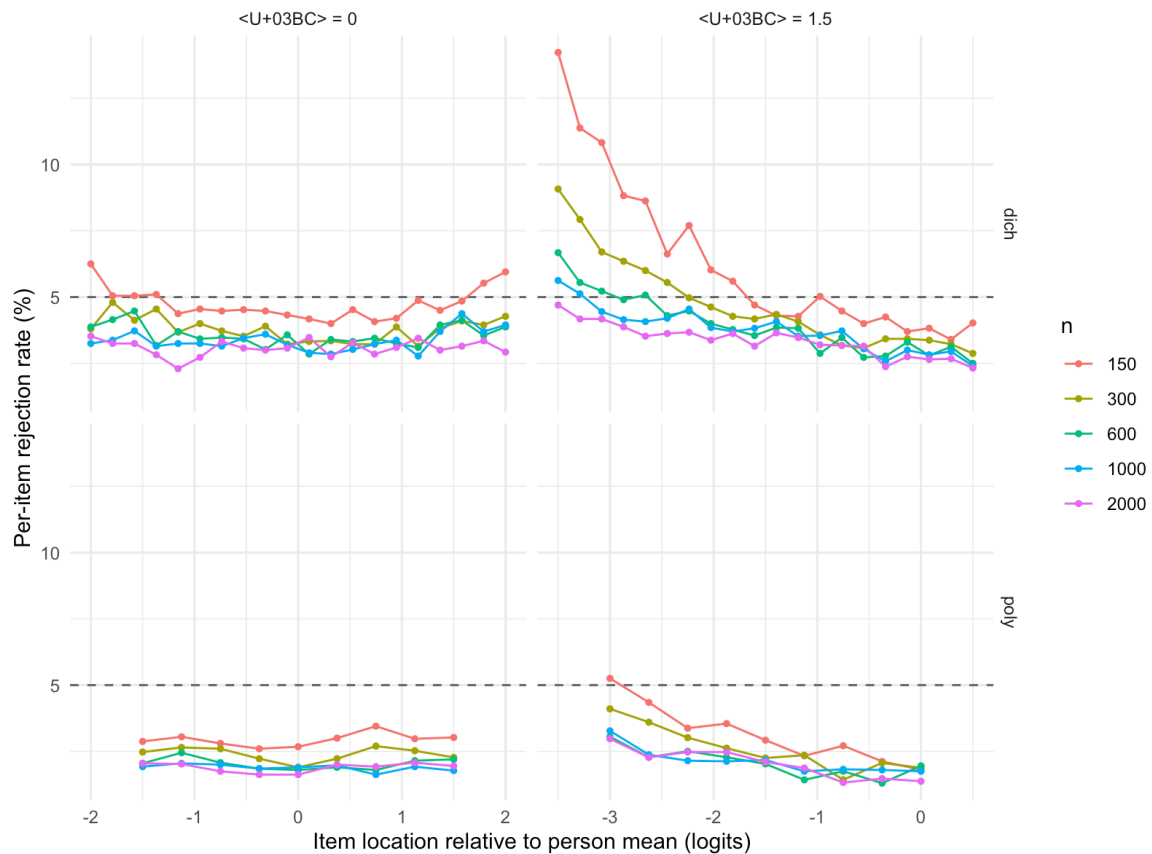
long |>
  group_by(set, mu, n, item, location) |>
  summarise(rate = 100 * mean(p < .05), .groups = "drop") |>
  mutate(rel_location = location - mu,

```

```

mu = paste("μ =", mu)) |>
ggplot(aes(rel_location, rate, colour = factor(n))) +
geom_hline(yintercept = 5, linetype = "dashed", colour = "grey40") +
geom_line() +
geom_point(size = 1) +
facet_grid(set ~ mu, scales = "free_x") +
labs(x = "Item location relative to person mean (logits)",
     y = "Per-item rejection rate (%)", colour = "n") +
theme_minimal()

```



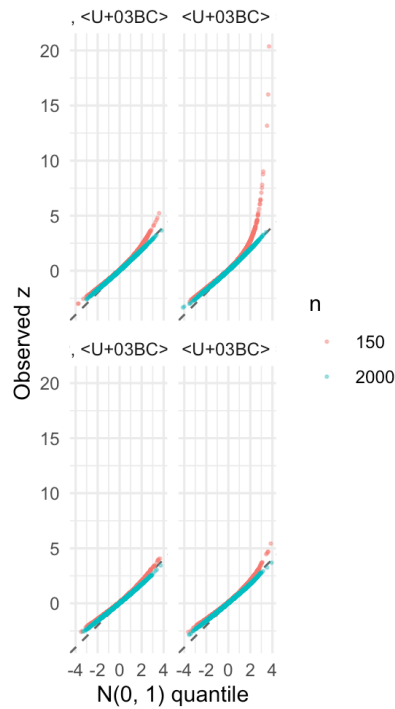
Distribution of z

Quantile plot against the standard normal, pooled over items, for the smallest and largest n. Points on the diagonal mean the reference distribution is right.

```

long |>
  filter(n %in% range(NS)) |>
  group_by(set, mu, n) |>
  mutate(theoretical = qnorm(ppoints(n()))[rank(z, ties.method = "first")]) |>
  slice_sample(n = 5000) |>
  ungroup() |>
  mutate(cell = paste0(set, " μ =", mu)) |>
  ggplot(aes(theoretical, z, colour = factor(n))) +
  geom_abline(linetype = "dashed", colour = "grey40") +
  geom_point(size = 0.4, alpha = 0.4) +
  facet_wrap(~cell) +
  coord_equal() +
  labs(x = "N(0, 1) quantile", y = "Observed z", colour = "n") +
  theme_minimal()

```



Interpretation

The asymptotic item-restscore test is not calibrated. Two separate distortions act on it, and neither vanishes as n grows.

1. The ASE is right for the observed gamma but wrong for the test statistic. The SD of the observed gamma across replications matches the root mean squared ASE to within 1 percent in every cell, so the delta-method SE is not the problem. The problem is that the expected gamma varies across samples and moves with the observed gamma (correlation about .41 for the dichotomous set and .55 for the polytomous set). Its variance roughly equals the covariance, so the difference varies less than the observed gamma alone. The SD of the difference is 0.91 times the ASE for dichotomous items and 0.83 times for polytomous items, nearly constant across sample size and targeting. Dividing by the ASE therefore shrinks z by a fixed factor, and the test is conservative at every sample size. This is point 1 from the Purpose section, and point 2 does not show up at all.

2. A small-sample bias pushes z upwards. Mean z is .15 at $n = 150$ and falls to .04 at $n = 2000$, roughly in proportion to $1/\sqrt{n}$. This is what a bias of order $1/n$ in the gamma difference produces. It inflates the upper tail (observed gamma above expected, labelled overfit by the package) and depletes the lower tail. It is largest for items far from the person mean, reaching 14 percent two-sided at $n = 150$ for the easiest dichotomous item under mistargeting.

Net effect on single items. Nominal is 5 percent two-sided. Dichotomous items reject 4.7 percent at $n = 150$ when targeted, falling to 3.1 at $n = 2000$. Polytomous items reject 2.9 falling to 1.9. The tails are lopsided. The lower tail, which is the underfit direction and the usual signature of multidimensionality, rejects 0.2 to 1.2 percent against a nominal 2.5 in every cell, a tenth to a half of what it should. The upper tail exceeds 2.5 percent at small n for dichotomous items. The power cost of the deflated lower tail is not measured here, but it follows directly: underfit has to be larger than the nominal level implies before it is flagged.

Family-wise rates, which is what a user sees. With BH over the items, the probability that a true Rasch model gets at least one item flagged is:

- dichotomous, targeted: 13.5, 6.2, 4.1, 2.7 and 2.3 percent at $n = 150$ to 2000
- dichotomous, mistargeted: 30.5, 13.9, 7.4, 4.9 and 3.2 percent
- polytomous, targeted: 4.2 falling to 0.9 to 1.1 percent

- polytomous, mistargeted: 5.1 falling to 1.2 to 1.6 percent

So the same test is liberal for dichotomous data at small n , especially when mistargeted, and conservative for polytomous data at every n . No single correction to the nominal level fixes both. BH and Bonferroni give nearly identical rates, as expected under a complete null.

Discarded replications. 28 percent of the mistargeted polytomous replications at $n = 150$ and 6 percent at $n = 300$ had an unobserved category and were discarded. Those two cells describe samples where every category is used, which are also the only samples the package can fit.

For easyRasch2. Both distortions come from treating the expected gamma as a fixed reference, and both depend on the item set, sample size and targeting. A parametric bootstrap that simulates from the fitted model, **re-estimates the thresholds in every replication**, and recomputes the observed minus expected difference would absorb both, in the same way as `RMitemInfitCutoff()`. A bootstrap that holds the thresholds fixed would reproduce distortion 1. The item-restscore statistic is cheap (about 0.1 s per replication including the CML fit), so an `RMitemRestscoreCutoff()` at the same iteration counts as the infit cutoffs is affordable. This simulation says nothing about power. That comparison, against infit with simulation-based cutoffs, is the natural next step.

Reference

van der Ark, L. A., & van Aert, R. C. M. (2015). Comparing confidence intervals for Goodman and Kruskal's gamma coefficient. *Journal of Statistical Computation and Simulation*, 85(12), 2491-2505.